

ProblemSet_6&7

Baibhab_Chowdhury_738

2026-03-19

Problem Set 6: Model Selection

Consider the “Hitters” data. For loading the data you need to install ISLR library. We wish to predict a baseball player’s salary on the basis of various predictors. So first we need to choose an optimal model.

```
# Load libraries
```

```
library(ISLR)
```

```
## Warning: package 'ISLR' was built under R version 4.4.3
```

```
library(leaps) # For subset selection
```

```
## Warning: package 'leaps' was built under R version 4.4.3
```

1. Clean the data by removing all rows that have missing value in any variable. You will be working with this clean data.

```
# Load data
```

```
data(Hitters)
```

```
# Remove rows with missing values
```

```
hitters <- na.omit(Hitters)
```

```
# Check dimensions
```

```
dim(hitters)
```

```
## [1] 263 20
```

Comments

The dataset initially contains some missing values in the Salary variable. These observations are removed using `na.omit()` to ensure that the analysis is performed only on complete cases.

After cleaning the dataset, the number of observations is reduced but the dataset becomes suitable for regression modelling without missing data issues.

2. Split the data randomly into train-test keeping 80% as training sample and remaining as test sample.

```
set.seed(123)
```

```
n <- nrow(hitters)
```

```
train_index <- sample(1:n, 0.8*n)
```

```
train <- hitters[train_index, ]  
test <- hitters[-train_index, ]
```

Comments

The dataset is randomly divided into:

80% training data: used to build the regression models. 20% test data: used to evaluate predictive performance.

Setting a seed ensures that the random split is reproducible.

3. Perform the best subset selection on the training sample. While choosing the best model fit up to a 19 variable model. Choose the best model according to adjusted R squared, BIC, AIC and Cp values. In case the best model differs according to the selection criteria, report the best fitted model according to each selection criterion.

```
best_model <- regsubsets(Salary ~ ., data=train, nvmax=19)
```

```
best_summary <- summary(best_model)
```

```
#Extract Selection Criteria
```

```
adjr2 <- best_summary$adjr2
```

```
cp <- best_summary$cp
```

```
bic <- best_summary$bic
```

```
# AIC calculation
```

```
rss <- best_summary$rss
```

```
n_train <- nrow(train)
```

```
p <- 1:19
```

```
aic <- n_train*log(rss/n_train) + 2*p
```

```
#Best Models According to Criteria
```

```
best_adjr2 <- which.max(adjr2)
```

```
best_cp <- which.min(cp)
```

```
best_bic <- which.min(bic)
```

```
best_aic <- which.min(aic)
```

```
best_adjr2
```

```
## [1] 12
```

```
best_cp
```

```
## [1] 11
```

```
best_bic
```

```
## [1] 5
```

```

best_aic
## [1] 12

#Variables in best models
coef(best_model, best_adjR2)

## (Intercept)          AtBat          Hits          Walks          CAtBat
CRuns
## 154.1594365 -1.8722264  5.7547106  6.3852096 -0.2092154
1.7712871
##          CRBI          CWalks  DivisionW          PutOuts          Assists
Errors
## 1.1419024 -0.8989348 -121.8633686  0.2013298  0.5363804 -
6.4726568
## NewLeagueN
## 75.8378982

coef(best_model, best_cp)

## (Intercept)          AtBat          Hits          Walks          CAtBat
CRuns
## 134.1593585 -1.9328699  5.8271814  6.5037245 -0.2063323
1.7820180
##          CRBI          CWalks  DivisionW          PutOuts          Assists
NewLeagueN
## 1.1120229 -0.8879140 -121.2135304  0.1925628  0.3316973
71.1433129

coef(best_model, best_bic)

## (Intercept)          Walks          CAtBat          CHits          CHmRun
DivisionW
## 149.5853608  5.9582020 -0.5241611  2.0687418  1.7652811 -
129.0165012

coef(best_model, best_aic)

## (Intercept)          AtBat          Hits          Walks          CAtBat
CRuns
## 154.1594365 -1.8722264  5.7547106  6.3852096 -0.2092154
1.7712871
##          CRBI          CWalks  DivisionW          PutOuts          Assists
Errors
## 1.1419024 -0.8989348 -121.8633686  0.2013298  0.5363804 -
6.4726568
## NewLeagueN
## 75.8378982

#Prediction function needed for Test MSE.
predict_regsubsets <- function(object, newdata, id){

```

```

form <- as.formula(object$call[[2]])
mat <- model.matrix(form, newdata)
coefi <- coef(object, id=id)
xvars <- names(coefi)

mat[,xvars] %*% coefi
}

```

Interpretation

Best subset selection evaluates all possible combinations of predictors and selects the best model for each model size. This method guarantees finding the optimal subset of predictors but can be computationally expensive.

Each model selection criterion identifies a potentially different optimal model. Adjusted R^2 selects the model that explains the largest proportion of variance while penalizing unnecessary predictors. Cp identifies models with low bias and variance. BIC heavily penalizes model complexity, favoring simpler models. AIC balances goodness of fit with model complexity.

If these criteria select different models, it indicates that there is a trade-off between model complexity and predictive accuracy.

4. Find the training MSE and test MSE for each of the best models selected according to different criterion as mentioned in 3 and compare their values.

```

calc_mse <- function(id){

train_pred <- predict_regrsubsets(best_model, train, id)
test_pred <- predict_regrsubsets(best_model, test, id)

train_mse <- mean((train$Salary - train_pred)^2)
test_mse <- mean((test$Salary - test_pred)^2)

return(c(train_mse, test_mse))
}

mse_adj2 <- calc_mse(best_adj2)
mse_cp <- calc_mse(best_cp)
mse_bic <- calc_mse(best_bic)
mse_aic <- calc_mse(best_aic)

mse_adj2
## [1] 88935.8 130066.9

mse_cp
## [1] 89790.63 125652.28

```



```
backward_sum <- summary(backward_model)

adjr2_b <- which.max(backward_sum$adjr2)
cp_b <- which.min(backward_sum$cp)
bic_b <- which.min(backward_sum$bic)

rss_b <- backward_sum$rss
aic_b <- n_train*log(rss_b/n_train) + 2*(1:19)
aic_b_best <- which.min(aic_b)
```

Backward MSE Calculation.

```
calc_mse_backward <- function(id){

train_pred <- predict_regrsubsets(backward_model, train, id)
test_pred <- predict_regrsubsets(backward_model, test, id)

c(mean((train$Salary-train_pred)^2),
  mean((test$Salary-test_pred)^2))
}

calc_mse_backward(adjr2_b)
## [1] 88865.02 134736.70

calc_mse_backward(cp_b)
## [1] 88865.02 134736.70

calc_mse_backward(bic_b)
## [1] 97026.43 153090.52

calc_mse_backward(aic_b_best)
## [1] 88865.02 134736.70
```

7. Compare the results of the three methods in terms of choice of best model, train MSE and test MSE.

Best Subset

Evaluates all possible models
Most computationally expensive
Often gives best fit

Forward Selection

Adds predictors gradually
Faster but may miss optimal model

Backward Selection

Starts with full model

Removes unnecessary predictors

Best subset selection evaluates all possible predictor combinations and generally produces the most optimal model according to statistical criteria such as Adjusted R^2 , BIC, AIC and Cp. Forward and backward stepwise selection are computationally efficient alternatives that approximate the best subset results. The final comparison of training and test MSE indicates which model provides the best predictive performance.

Problem Set 7: Shrinkage Methods

Comparison of least squares, ridge regression, LASSO and Elastic Net

You would be working with Hitters data set available in the ISLR library. You have to predict the Salary based on all available variables. First make sure that you have deleted all rows with one or more missing values.

```
rm(list=ls())
library(ISLR)
data(Hitters)

# remove missing values
hitters = na.omit(Hitters)
dim(hitters)

## [1] 263 20
```

1. Split the data set into a training set and a test set randomly keeping 100 observations in test set and remaining in the training set.

```
set.seed(123)

n = nrow(hitters)

test_index = sample(1:n, 100)

train = hitters[-test_index, ]
test = hitters[test_index, ]
```

2. Train the model using least squares and report the train MSE and test MSE.

```
library(glmnet)

## Warning: package 'glmnet' was built under R version 4.4.3
## Loading required package: Matrix
## Loaded glmnet 4.1-10
```

```

#model.matrix() converts categorical variables into dummy variables
#[, -1] removes intercept column
x_train = model.matrix(Salary ~ ., train)[, -1]
y_train = train$Salary

x_test = model.matrix(Salary ~ ., test)[, -1]
y_test = test$Salary

#Ridge Model
lm_model = lm(Salary ~ ., data = train)

train_pred = predict(lm_model, train)
test_pred = predict(lm_model, test)

train_mse_lm = mean((train$Salary - train_pred)^2)
test_mse_lm = mean((test$Salary - test_pred)^2)

train_mse_lm
## [1] 105334.2

test_mse_lm
## [1] 87803.91

```

Train MSE → model fit

Test MSE → prediction accuracy

3. Fit ridge regression models on the training set, with grid values of λ . Report the following:

```

lambda_grid = 10^seq(5, -2, length = 100)

ridge_model = glmnet(x_train, y_train, alpha = 0, lambda = lambda_grid)

```

Explanation:

alpha = 0 → Ridge regression

lambda_grid → penalty values

(i) the dimension of the coefficient matrix so obtained.

```

dim(coef(ridge_model))
## [1] 20 100

```

Interpretation:

rows → predictors + intercept

columns → number of lambda values

- (ii) The l2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ . Comment on the output.

```
coef50 = coef(ridge_model, s = lambda_grid[50])
coef60 = coef(ridge_model, s = lambda_grid[60])

norm50 = sqrt(sum(coef50[-1]^2))
norm60 = sqrt(sum(coef60[-1]^2))

norm50
## [1] 155.9135

norm60
## [1] 165.5575
```

Larger λ → stronger shrinkage → smaller L2 norm

- (iii) Test MSE corresponding to these two choices of λ .

```
pred50 = predict(ridge_model, s = lambda_grid[50], newx = x_test)
pred60 = predict(ridge_model, s = lambda_grid[60], newx = x_test)

mse50 = mean((y_test - pred50)^2)
mse60 = mean((y_test - pred60)^2)

mse50
## [1] 89317.28

mse60
## [1] 87580.18
```